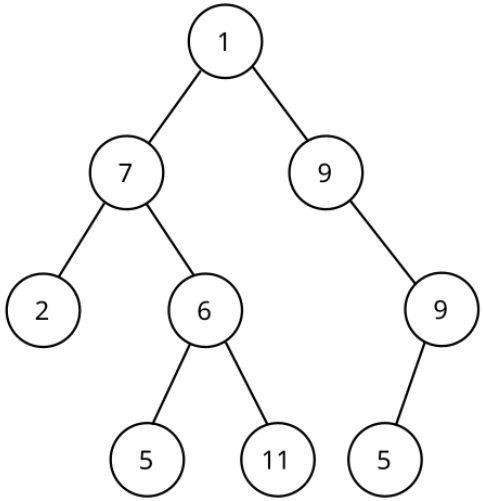
Lecture 19

Searching and Sorting II



Announcements

- Office hours are up on course website
  - T: 12pm-4pm; F: 11am-1pm
  - Schedule an appointment individually or as a group

- Project II deliverable 1 due 11/13

Last time

Searching through a list

- Linear and binary search

Searching through a graph (lab)

- Dijkstra’s algorithm

Sorting a list

- Selection sort

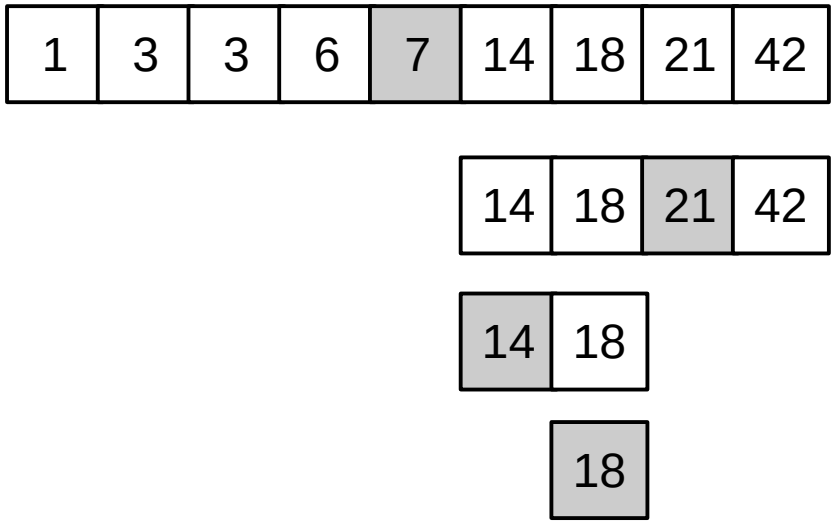
Today

Sorting algorithms

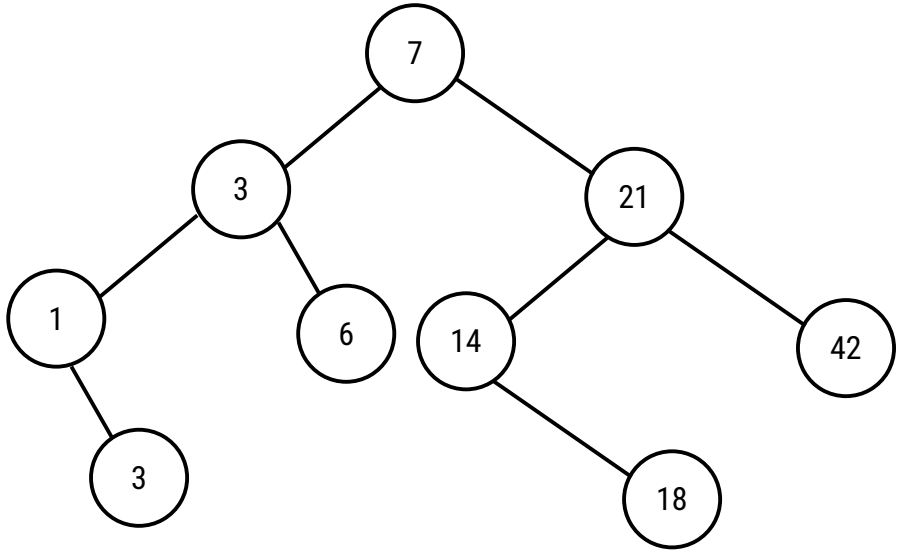
- Merge sort

Binary Tree

- Is 19 in the list? No, return `-1`



- Represent binary search as a tree



Running time: $\Theta(\log n)$

Tree height: $3 = \lceil \log n \rceil \leq \log n + 1$

Selection Sort

Initialize a pointer at index 0

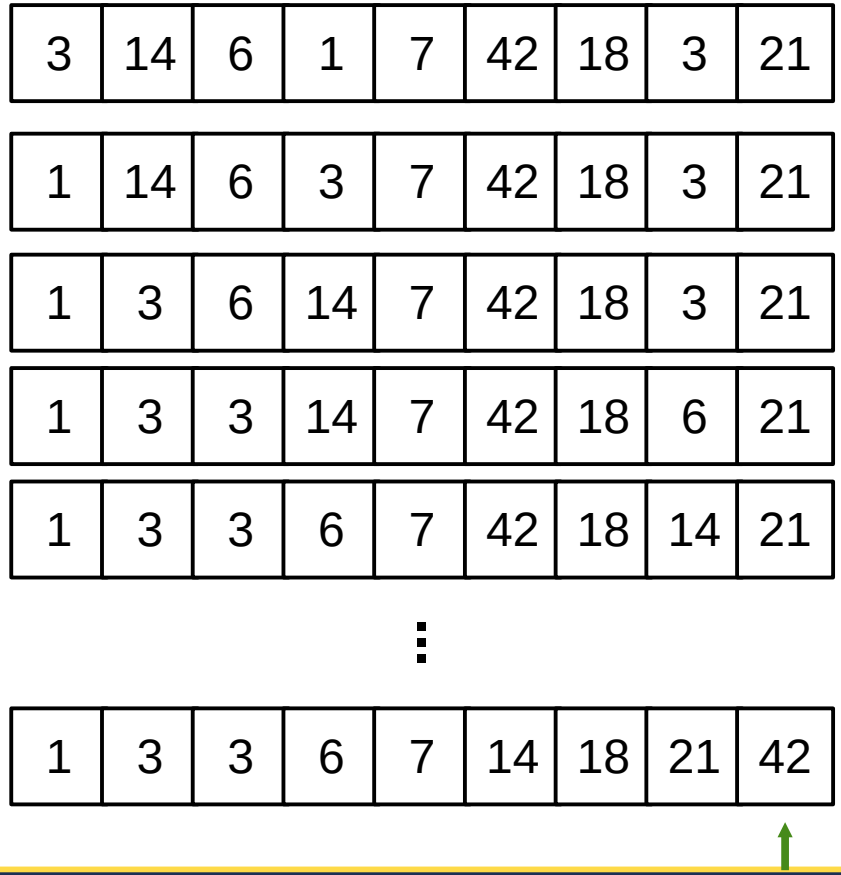
- Left of the pointer is sorted

Find min value from pointer

- Swap with pointer value
- Increment pointer

Continue until pointer reaches end of list

$\Theta(n^2)$ running time



Recap

Searching an arbitrary list: $O(n)$

Searching a sorted list: $O(\log n)$

Selection sort: $O(n^2)$

Q: can we beat linear search by sorting first, then running binary search?

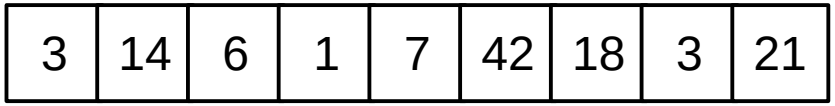
- Not yet: selection sort + binary search is $O(n^2) + O(\log n) = O(n^2)$

Merge Sort

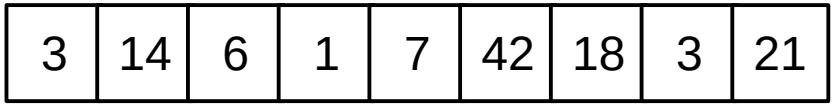
Idea: recursively sort L

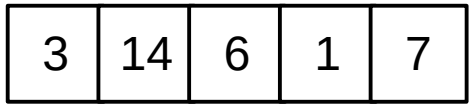
```
MergeSort(L):
    if L has one item do
        return L
    Divide L into two halves A, B
    A ← MergeSort(A)
    B ← MergeSort(B)
    L ← Merge(A, B)
    return L
```

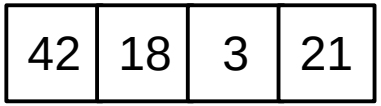
Recursively Decompose



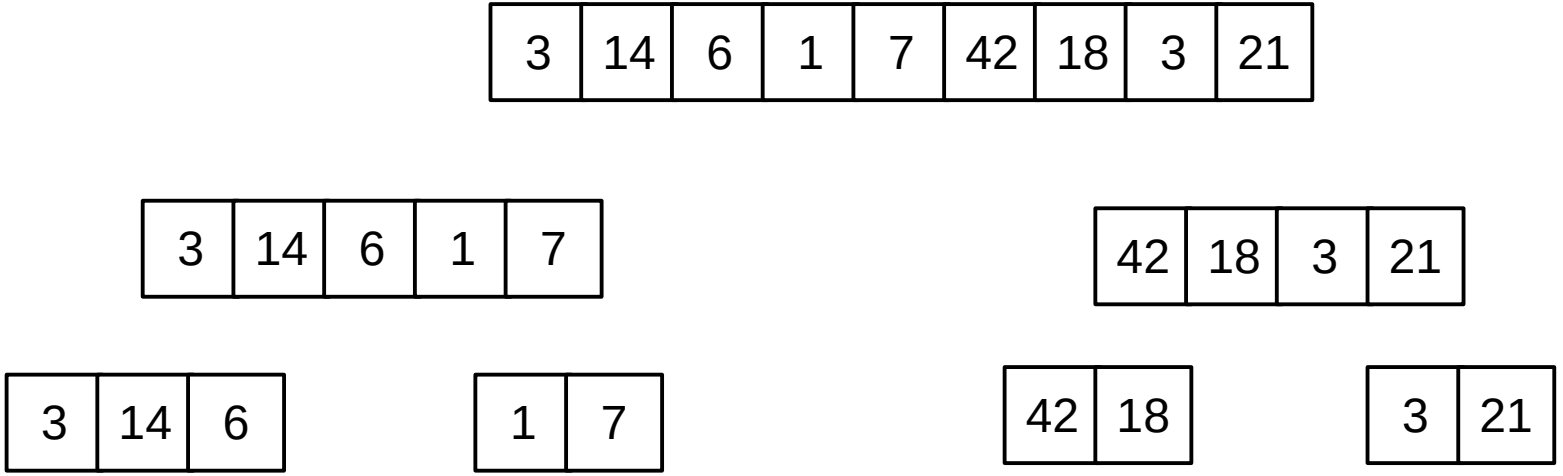
Recursively Decompose



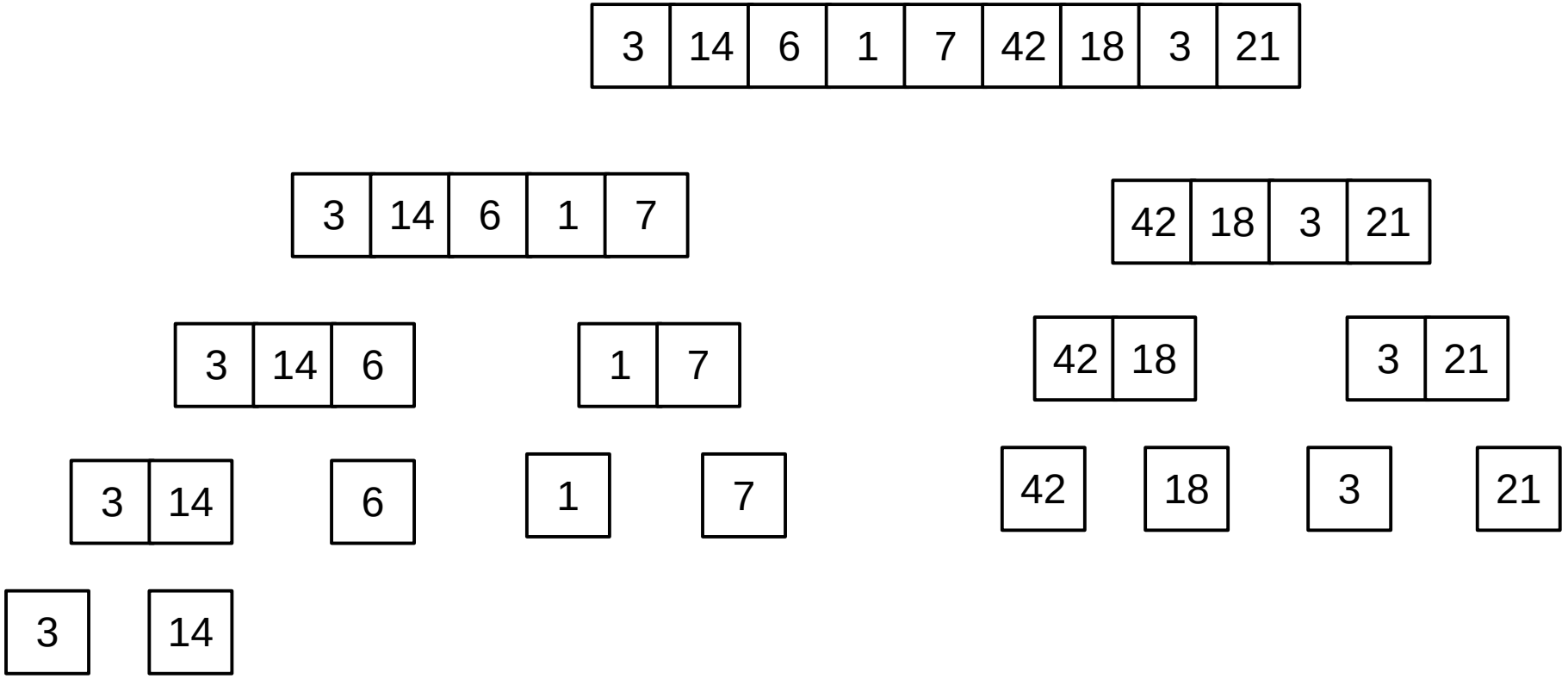




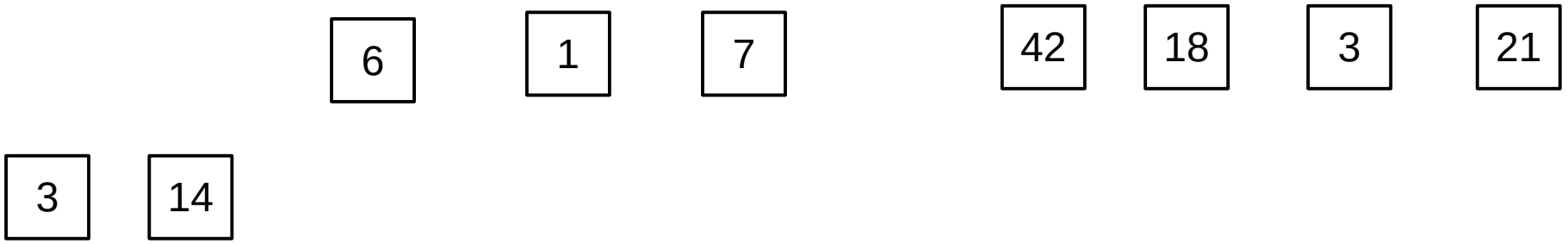
Recursively Decompose



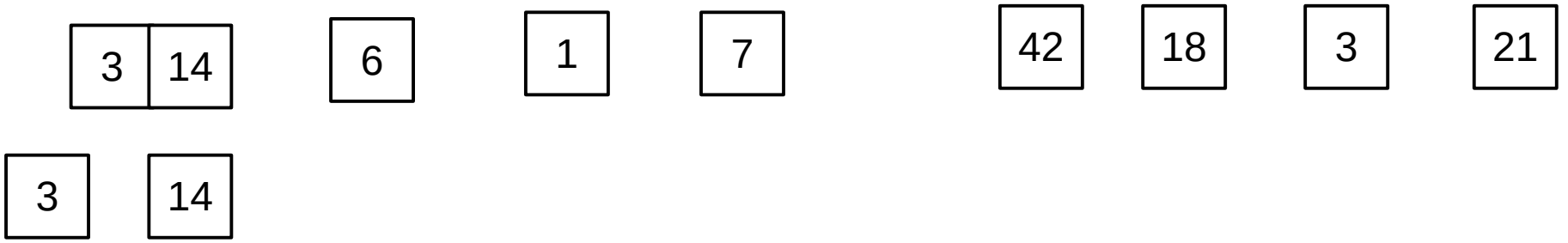
Recursively Decompose



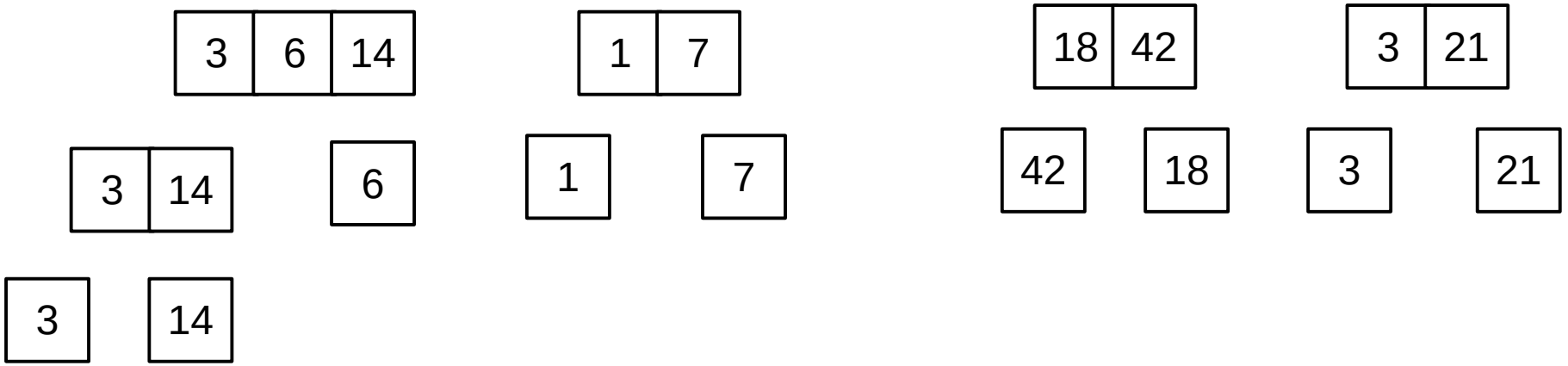
Reconstruct



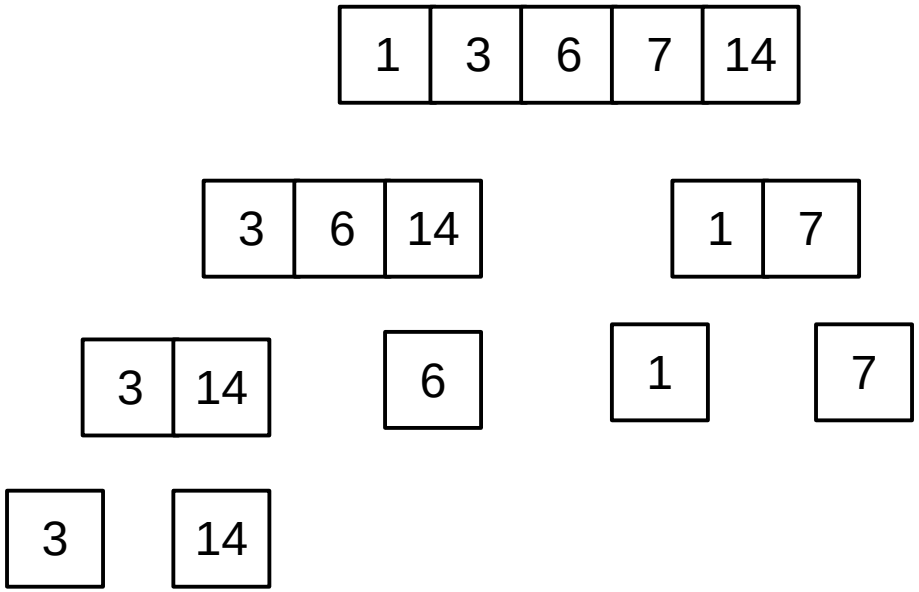
Reconstruct

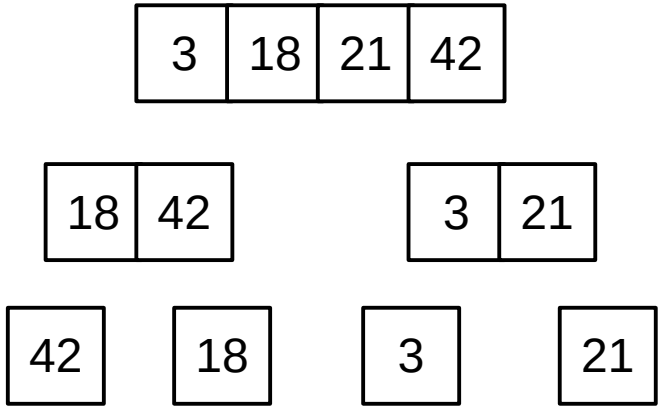


Reconstruct

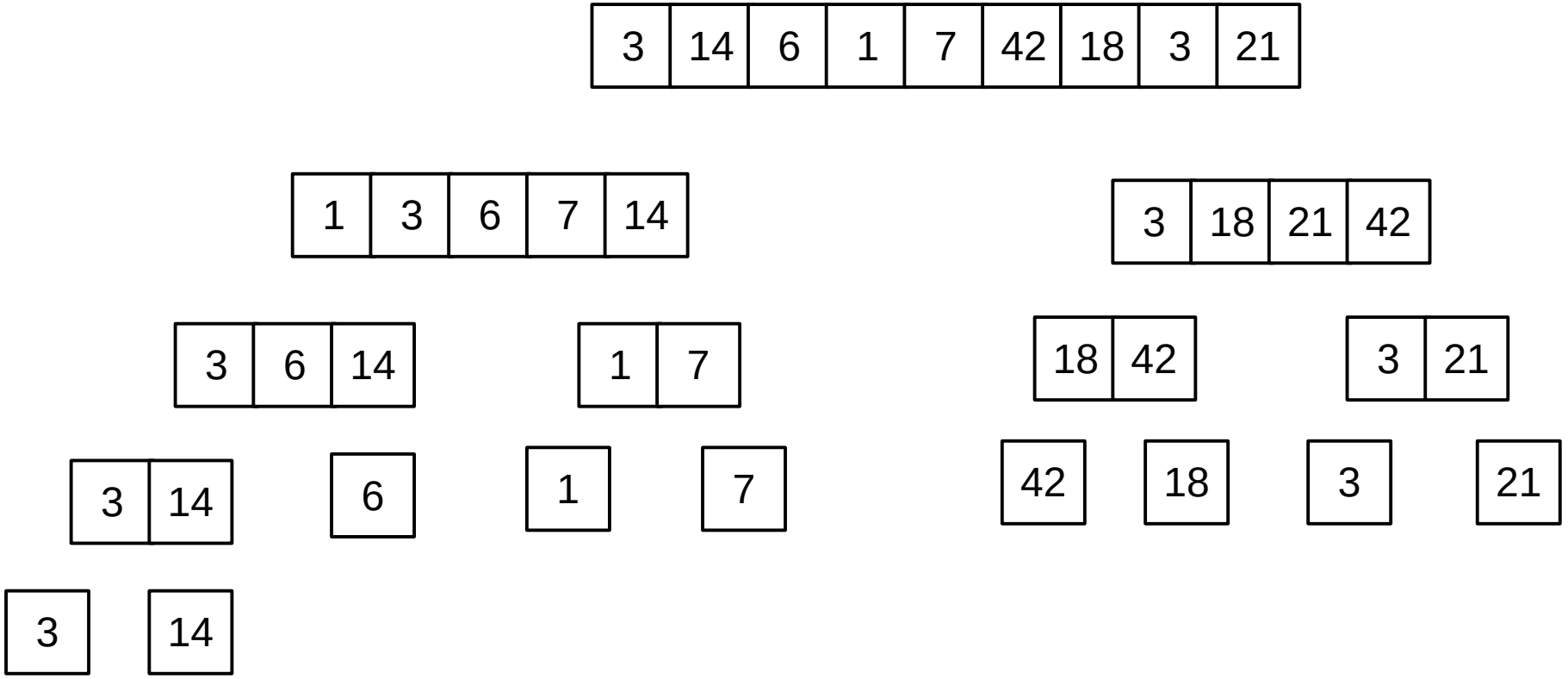


Reconstruct





Reconstruct



Efficient Reconstruction

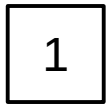
Theorem: If A, B are sorted lists, then can we can concatenate and sort them in linear time



Efficient Reconstruction

Theorem: If A, B are sorted lists, then can we can concatenate and sort them in linear time

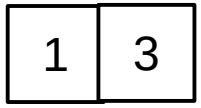




Efficient Reconstruction

Theorem: If A, B are sorted lists, then can we can concatenate and sort them in linear time

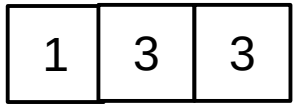




Efficient Reconstruction

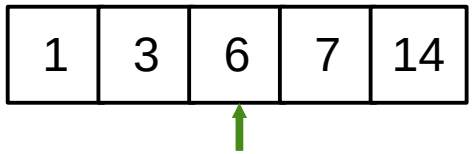
Theorem: If A, B are sorted lists, then can we can concatenate and sort them in linear time

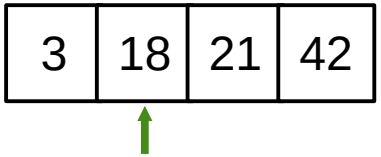


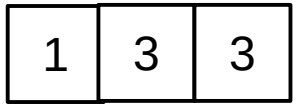


Efficient Reconstruction

Theorem: If A, B are sorted lists, then can we can concatenate and sort them in linear time







Continue until both A and B are empty

Requires only n comparisons

Merge Sort

Idea: recursively sort L

base case: a single item is sorted `O(1)`

```
MergeSort(L):
    if L has one item do
        return L
    Divide L into two halves A, B
    A ← MergeSort(A)
    B ← MergeSort(B)
    L ← Merge(A, B)
    return L
```

Merge Sort

Idea: recursively sort L

```
MergeSort(L):
    if L has one item do
        return L
    Divide L into two halves A, B
    A ← MergeSort(A)
    B ← MergeSort(B)
    L ← Merge(A, B)
    return L
```

base case: a single item is sorted $O(1)$

recursive cases

Merge Sort

Idea: recursively sort L

```
MergeSort(L):
    if L has one item do
        return L
    Divide L into two halves A, B
    A ← MergeSort(A)
    B ← MergeSort(B)
    L ← Merge(A, B)
    return L
```

base case: a single item is sorted `O(1)`

recursive cases

combine sorted lists into a sorted list `O(n)`

Merge Sort

Let `T(n)` be the running time of merge sort

Idea: recursively sort L

```
MergeSort(L):
    if L has one item do
        return L
    Divide L into two halves A, B
    A ← MergeSort(A)
    B ← MergeSort(B)
    L ← Merge(A, B)
    return L
```

base case: a single item is sorted `O(1)`

recursive cases

combine sorted lists into a sorted list `O(n)`

Merge Sort

Let `T(n)` be the running time of merge sort

Idea: recursively sort L

```
MergeSort(L):
    if L has one item do
        return L
    Divide L into two halves A, B
    A ← MergeSort(A)
    B ← MergeSort(B)
    L ← Merge(A, B)
    return L
```

base case: a single item is sorted `O(1)`

recursive cases `T(n/2)`

combine sorted lists into a sorted list `O(n)`

Merge Sort

Let T(n) be the running time of merge sort

Idea: recursively sort L

base case: a single item is sorted O(1)

```
MergeSort(L):
    if L has one item do
        return L
    Divide L into two halves A, B
    A ← MergeSort(A)
    B ← MergeSort(B)
    L ← Merge(A, B)
    return L
```

recursive cases T(n/2)

combine sorted lists into a sorted list O(n)

$$T(n) = 2T(n/2) + O(n)$$

Recurrence

Recurrence

- `T(n)` is defined in terms of smaller values of n
- Base case: `T(n)` is `O(1)`

Goal: *solve* the recurrence by finding a simple expression for `T(n)`

Recurrence

Recurrence relation

- $T(n) = 2T(n/2) + O(n)$
- $T(1) = O(1)$

Let's use the definition of Big-O

- $T(n) \leq 2T(n/2) + cn$
- $T(1) \leq c$

Recurrence

Recurrence relation

- $T(n) = 2T(n/2) + O(n)$
- $T(1) = O(1)$

Let's use the definition of Big-O

- $T(n) \leq 2T(n/2) + cn$
- $T(1) \leq c$

Q: Why can we use the same value of c for both instances of the Big-O definition?

a) Take $c = \min(c_1, c_2)$ where c_1 and c_2 are the values from each instance

b) Take $c = \max(c_1, c_2)$ where c_1 and c_2 are the values from each instance

c) It's not okay!

Recurrence

Recurrence relation

- $T(n) = 2T(n/2) + O(n)$
- $T(1) = O(1)$

Let's use the definition of Big-O

- $T(n) \leq 2T(n/2) + cn$
- $T(1) \leq c$

Q: Why can we use the same value of c for both instances of the Big-O definition?

a) Take $c = \min(c_1, c_2)$ where c_1 and c_2 are the values from each instance

b) Take $c = \max(c_1, c_2)$ where c_1 and c_2 are the values from each instance

c) It's not okay!

Recurrence

Recurrence relation

- $T(n) = 2T(n/2) + O(n)$
- $T(1) = O(1)$

Let's use the definition of Big-O

- $T(n) \leq 2T(n/2) + cn$
- $T(1) \leq c$

Strategies for Solving

- Unrolling recursion
- Recursion tree

Unrolling

$T(n) \leq 2T(n/2) + cn$

Unrolling

$$
\begin{aligned}
T(n) &\leq 2T(n/2) + cn \\
&\leq 2\,(2T(n/4) + cn/2) + cn \\
&= 4\,T(n/4) + 2cn
\end{aligned}
$$

Unrolling

$$
\begin{aligned}
T(n) &\leq 2T(n/2) + cn \\
&\leq 2\,(2T(n/4) + cn/2) + cn \\
&= 4\,T(n/4) + 2cn \\
&\leq 4\,(2T(n/8) + cn/4) + 2cn \\
&= 8T(n/8) + 3cn
\end{aligned}
$$

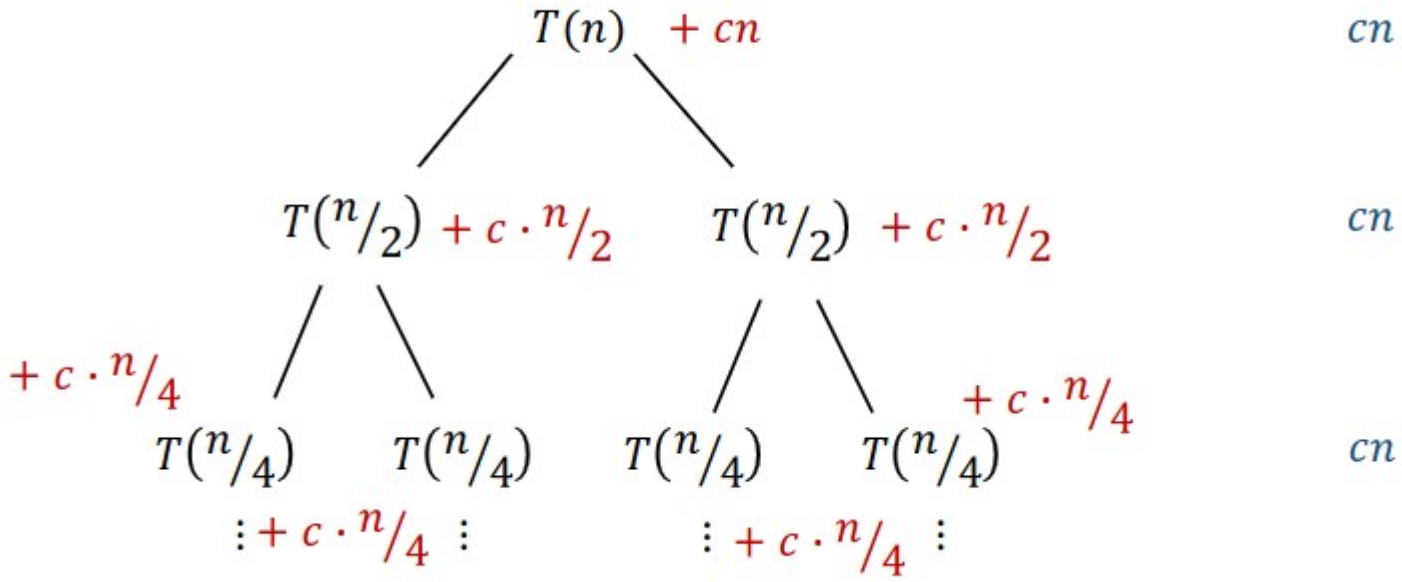
Unrolling

$$
\begin{aligned}
T(n) &\leq 2T(n/2) + cn \\
&\leq 2\,(2T(n/4) + cn/2) + cn \\
&= 4\,T(n/4) + 2cn \\
&\leq 4\,(2T(n/8) + cn/4) + 2cn \\
&= 8T(n/8) + 3cn \\
&\dots \\
&\leq n\,T(1) + \log n * cn
\end{aligned}
$$

Unrolling

$$
\begin{aligned}
T(n) &\leq 2T(n/2) + cn \\
&\leq 2\,(2T(n/4) + cn/2) + cn \\
&= 4\,T(n/4) + 2cn \\
&\leq 4\,(2T(n/8) + cn/4) + 2cn \\
&= 8T(n/8) + 3cn \\
&\dots \\
&\leq n\,T(1) + \log n * cn \\
&= cn + \log n * cn \\
&= O(n \log n)
\end{aligned}
$$

Recursion Tree



`log n + 1` levels each with `cn` work implies O(n log n)

Recap

Searching an arbitrary list: $O(n)$

Searching a sorted list: $O(\log n)$

Selection sort: $O(n^2)$

Merge sort: $O(n \log n)$

Q: can we beat linear search by sorting first, then running binary search?

- Nope: merge sort + binary search is $O(n \log n) + O(\log n) = O(n \log n)$

Next Time

- Divide and conquer algorithms